

Semana 2 — Clase Práctica en Laboratorio

Algoritmos con Condicionales, Aritmética Modular y Construcción de Cadenas

Introducción

Los problemas de esta semana representan un salto cualitativo respecto a la Semana 1: incorporan **estructuras condicionales** (`if / elif / else`), **aritmética modular** (operadores `//` y `%`) y **construcción progresiva de cadenas** (`str`), adelantando contenidos de la Semana 3. Cada problema sigue requiriendo la aplicación rigurosa de la **metodología de Pólya** y el **modelo de programa** del curso.

Los cuatro problemas están diseñados con **conexiones deliberadas**: los Problemas 1 y 2 trabajan con fechas en formato AAAAMMDD (conectando con el Problema 3 de la Semana 1), mientras que los Problemas 3 y 4 exploran la conversión entre sistemas de representación numérica, ejercitando intensivamente la división entera y el resto.

Nota sobre el adelanto de condicionales

Las estructuras `if / elif / else` se presentan formalmente en la Semana 3. En esta práctica se introducen de manera natural, motivadas por la necesidad de resolver problemas que requieren tomar decisiones.

Sintaxis estructuras de decisión: `if condición / elif condición / else`

Operadores de comparación: `<`, `>`, `<=`, `>=`, `==`, `!=`

Problema 1: Día de la semana por la Congruencia de Zeller

Enunciado

Escribir un programa en Python que solicite al usuario una fecha en formato **AAAAMMDD** (un único entero, como los producidos en el Problema 3 de la Semana 1), verifique que corresponda a una fecha válida a partir de la vigencia del **Calendario Gregoriano** (15 de octubre de 1582), y determine a qué **día de la semana** corresponde, mostrándolo por su nombre en español.

Fundamento matemático

La **Congruencia de Zeller**, publicada por el matemático alemán Christian Zeller en 1882, es una fórmula que calcula el día de la semana para cualquier fecha del calendario gregoriano. Su elegancia radica en que reduce el problema a pura aritmética modular, sin requerir tablas de consulta.

Extracción de los componentes de la fecha

A partir del entero AAAAMMDD se extraen año, mes y día usando división entera (//) y resto (%):

$$\text{anio} = \text{fecha} // 10000 \quad \text{mes} = (\text{fecha} // 100) \% 100 \quad \text{dia} = \text{fecha} \% 100$$

Ajuste de Zeller para enero y febrero

La fórmula de Zeller trata enero y febrero como los meses 13 y 14 del año anterior. Esto simplifica el cálculo porque los días adicionales del año bisiesto (29 de febrero) quedan al final del «año de Zeller»:

```
# Ajuste: si mes es 1 (enero) o 2 (febrero):
#   mes = mes + 12
#   anio = anio - 1
```

Variables intermedias

$$K = \text{anio} \% 100 \quad (\text{año dentro del siglo})$$

$$J = \text{anio} // 100 \quad (\text{siglo})$$

Fórmula de Zeller

$$h = (\text{dia} + \lfloor 13(\text{mes}+1)/5 \rfloor + K + \lfloor K/4 \rfloor + \lfloor J/4 \rfloor - 2J) \% 7$$

donde $\lfloor x \rfloor$ denota la función piso (// en Python) y $\% 7$ es el resto de la división por 7. El resultado **h** codifica el día de la semana como: 0 = sábado, 1 = domingo, 2 = lunes, 3 = martes, 4 = miércoles, 5 = jueves, 6 = viernes.

En Python, el operador $\%$ con divisor positivo siempre devuelve un resultado no negativo, por lo que no es necesario ajustar valores negativos intermedios. Esta es una diferencia importante respecto a otros lenguajes (C, Java) donde $\%$ puede devolver negativos.

Ejemplos (casos de análisis y prueba)

Fecha (AAAAMMDD)	Día	Mes	Año	h	Día de semana	Observación
15821015	15	10	1582	6	Viernes	Primer día del Gregoriano
20250313	13	3	2025	5	Jueves	Fecha de referencia
20000101	1	1	2000	0	Sábado	Inicio de milenio
19691020	20	10	1969	2	Lunes	Llegada a la Luna (Apollo 11: jul)
20240229	29	2	2024	5	Jueves	Bisiesto
19000228	28	2	1900	4	Miércoles	1900 no es bisiesto

Verificación manual para el 13/03/2025

```

fecha = 20250313
anio = 20250313 // 10000      = 2025
mes = (20250313 // 100) % 100 = 3
dia = 20250313 % 100          = 13
# mes >= 3, no se ajusta
K = 2025 % 100 = 25
J = 2025 // 100 = 20
h = (13 + (13*4)//5 + 25 + 25//4 + 20//4 - 2*20) % 7
  = (13 + 10 + 25 + 6 + 5 - 40) % 7
  = 19 % 7 = 5 → Jueves ✓

```

Orientaciones para la resolución

Análisis: Una entrada `int` (fecha en formato AAAAMMDD), extracción de componentes, ajuste condicional para enero/febrero (`if mes <= 2`), cálculo de la fórmula, y traducción del resultado numérico al nombre del día (cadena de decisiones `if/elif/else`).

Diseño: Variables: `fecha` (`int`, entrada); `anio`, `mes`, `dia`, `K`, `J`, `h` (`int`, intermedias); `nombre_dia` (`str`, resultado). La traducción de `h` a nombre requiere una cadena de `if/elif` con 7 ramas.

Codificación: Notar que `13*(mes+1)//5` y no `13*(mes+1)/5`: se necesita división entera (piso). También puede usarse una cadena indexada como `'sábado domingo lunes martes miércoles jueves viernes'.split()` y acceder con `[h]` para evitar la cadena de `if/elif` (anticipo de listas, Semana 8).

Reflexión: el operador % en distintos lenguajes

Python garantiza que $a \% b$ tiene el mismo signo que b cuando $b > 0$. En C y Java, el resultado puede ser negativo si a es negativo: $(-19) \% 7$ da -5 en C pero 2 en Python. Esta diferencia es fuente frecuente de errores al trasladar algoritmos entre lenguajes. La Congruencia de Zeller funciona directamente en Python gracias a este comportamiento.

Conexión con el Calendario Gregoriano

El calendario gregoriano corrige el desfase del juliano eliminando 3 días bisiestos cada 400 años: los años divisibles por 100 no son bisiestos, excepto los divisibles por 400. Por eso 1900 no fue bisiesto pero 2000 sí. Los términos $[K/4]$ y $[J/4] - 2J$ en la fórmula de Zeller codifican exactamente esta regla. Verificarlo con los ejemplos de 1900 y 2000.

Problema 2: Diferencia entre dos fechas en años, meses y días

Enunciado

Escribir un programa en Python que solicite al usuario dos fechas en formato **AAAAMMDD** (enteros), determine cuál es la mayor y cuál la menor, y calcule la **diferencia expresada en años, meses y días**. Por ejemplo, la diferencia entre el 15/01/2023 y el 20/03/2025 es **2 años, 2 meses y 5 días**. El programa debe mostrar el resultado en formato legible.

Fundamento matemático

A diferencia de la resta en días (que se resuelve con una sola fórmula como el JDN¹), expresar la diferencia en años, meses y días requiere un **algoritmo con correcciones condicionales**, porque los meses tienen duraciones distintas y los años bisiestos alteran febrero. Se presentan dos enfoques de resolución.

Extracción de componentes (común a ambos enfoques)

```
anio = fecha // 10000    mes = (fecha // 100) % 100    dia = fecha % 100
```

Determinación de la fecha mayor

Gracias a la codificación AAAAMMDD, la comparación numérica directa determina el orden cronológico: si `fecha1 > fecha2`, entonces `fecha1` es posterior. El programa debe asegurar que la resta se realice siempre como **mayor – menor** para obtener componentes no negativos.

Días de cada mes (necesario para las correcciones)

Ambos enfoques necesitan conocer cuántos días tiene un mes dado. Las reglas son:

```
# Meses de 31 días: enero(1), marzo(3), mayo(5), julio(7), agosto(8), octubre(10),
#                   diciembre(12)
# Meses de 30 días: abril(4), junio(6), septiembre(9), noviembre(11)
# Febrero: 29 si bisiesto, 28 si no
# Año bisiesto: se puede dividir entre 4 pero no entre 100, o bien entre 400
# Ejemplo: 2024 bisiesto, 1900 no, 2000 sí
```

¹ El **Número de Día Juliano** (JDN) es un conteo continuo de días desde el 1 de enero de 4713 a.C. (calendario juliano), ampliamente usado en astronomía y computación: $JDN = dia + [(153 \times (mes - 3) + 2) / 5] + 365 \times anio + [anio/4] - [anio/100] + [anio/400] + 1720997$

Enfoque A: Descomponer primero, restar después

Este enfoque trabaja directamente con los componentes (año, mes, día) de cada fecha. La idea es realizar una **resta componente a componente** (día–día, mes–mes, año–año) y luego aplicar **correcciones por préstamo** cuando un componente resulte negativo, similar a la resta con «llevo uno» de la aritmética escolar.

Paso 1: Resta directa de componentes

```
dif_dia = dia_mayor - dia_menor
dif_mes = mes_mayor - mes_menor
dif_anio = anio_mayor - anio_menor
```

Paso 2: Corrección de días (préstamo de un mes)

Si $\text{dif_dia} < 0$, el día de la fecha mayor es menor que el de la fecha menor (por ejemplo, del 25/01 al 10/03). En este caso se «pide prestado» un mes: se suman los días del mes anterior al mes mayor, y se resta 1 al mes:

```
if dif_dia < 0:
    # Sumar los días del mes anterior al de la fecha mayor
    mes_anterior = mes_mayor - 1 # (o 12 si mes_mayor es 1)
    dif_dia = dif_dia + dias_del_mes(mes_anterior, anio_mayor)
    dif_mes = dif_mes - 1
```

Paso 3: Corrección de meses (préstamo de un año)

Si $\text{dif_mes} < 0$ (puede ocurrir originalmente o tras la corrección del paso 2), se «pide prestado» un año:

```
if dif_mes < 0:
    dif_mes = dif_mes + 12
    dif_anio = dif_anio - 1
```

Ejemplo paso a paso: del 25/01/2023 al 10/03/2025

```
# Componentes
dia_mayor=10, mes_mayor=3, anio_mayor=2025
dia_menor=25, mes_menor=1, anio_menor=2023

# Paso 1: Resta directa
dif_dia = 10 - 25 = -15
dif_mes = 3 - 1 = 2
dif_anio = 2025 - 2023 = 2

# Paso 2: dif_dia < 0 → pedir prestado un mes
# mes_anterior = 3 - 1 = 2 (febrero de 2025)
# 2025 no es bisiesto → febrero tiene 28 días
# dif_dia = -15 + 28 = 13
```

```
# dif_mes = 2 - 1 = 1

# Paso 3: dif_mes >= 0 → no se corrige

# Resultado: 2 años, 1 mes y 13 días ✓
```

Enfoque B: Restar primero, descomponer después

Este enfoque opera directamente sobre los números AAAAMMDD como enteros, explotando que la codificación posicional permite **restar las fechas como números** y luego corregir los componentes que resulten fuera de rango. Las correcciones usan el **complemento a 100** (para días y meses, que ocupan dos dígitos cada uno en AAAAMMDD).

Paso 1: Resta directa de enteros

$$\text{diferencia} = \text{fecha_mayor} - \text{fecha_menor}$$

Por ejemplo: $20250310 - 20230125 = 20185$.

Paso 2: Extracción de componentes de la diferencia

$$\begin{aligned}\text{dif_dia} &= \text{diferencia} \% 100 \\ \text{dif_mes} &= (\text{diferencia} // 100) \% 100 \\ \text{dif_anio} &= \text{diferencia} // 10000\end{aligned}$$

Paso 3: Correcciones con complemento a 100

Cuando el día o mes de la fecha menor es mayor que el correspondiente de la fecha mayor, la resta posicional produce un componente que parece válido pero está en **complemento a 100** (porque la resta «tomó prestado» del dígito superior). El valor correcto se obtiene restando de 100 y ajustando el componente superior:

```
# Si dif_dia está en complemento (supera los días del mes):
# El indicador es que dif_dia > dias_del_mes, o equivalentemente,
# que dia_menor > dia_mayor (la resta posicional pidió prestado)
if dia_menor > dia_mayor:
    dif_dia = 100 - dif_dia # complemento a 100 da el préstamo posicional
    # Pero necesitamos ajustar por los días reales del mes
    dif_dia = dias_del_mes(mes_anterior, anio_mayor) - (dia_menor - dia_mayor)
    dif_mes = dif_mes - 1

# Análogamente para meses:
if dif_mes < 0 or (mes_menor > mes_mayor adjusted):
    dif_mes = dif_mes + 12
    dif_anio = dif_anio - 1
```

La idea del complemento a 100 es que al restar AAAAMMDD como enteros, cada «préstamo» entre dígitos funciona en base 100 (porque cada componente ocupa dos dígitos), no en base 10. Así, si $\text{dia_mayor} = 10$ y $\text{dia_menor} = 25$, la resta posicional produce $\text{dif_dia} = 85$ (que es $100 - 15$), cuando el valor correcto depende de cuántos días tiene el mes prestador.

Ejemplo paso a paso: del 25/01/2023 al 10/03/2025

```
fecha_mayor = 20250310
fecha_menor = 20230125
```

```
# Paso 1: Resta directa
diferencia = 20250310 - 20230125 = 20185

# Paso 2: Extracción
dif_dia = 20185 % 100          = 85
dif_mes = (20185 // 100) % 100 = 1
dif_anio = 20185 // 10000      = 2

# Paso 3: Correcciones
# dia_menor(25) > dia_mayor(10) → hubo préstamo posicional
# dif_dia = 85 es el complemento a 100 de 15
# Pero la corrección real usa los días del mes anterior:
#   mes_anterior a marzo = febrero 2025 = 28 días
#   dif_dia = 28 - (25 - 10) = 28 - 15 = 13
#   dif_mes = 1 - 1 = 0 (el préstamo ya se descontó en la resta)
# PERO: la resta ya lo incorporó como base 100, el dif_mes
#   extraído (1) ya refleja el préstamo. Verificar:
#   mes_mayor(3) - mes_menor(1) = 2, pero con préstamo = 1

# Resultado: 2 años, 1 mes y 13 días ✓
```

Ejemplos completos (casos de análisis y prueba)

Fecha menor	Fecha mayor	Años	Meses	Días	Observación
20250101	20250313	0	2	12	Mismo año, sin préstamo de días
20230115	20250320	2	2	5	Caso sin correcciones
20230125	20250310	2	1	13	Préstamo de días (feb 28 d.)
20241125	20250310	0	3	13	Préstamo de días (feb 28 d.)
20230320	20250115	1	9	25	Préstamo de días y meses
20240229	20250228	0	11	30	Desde bisiesto a no bisiesto
20250313	20250313	0	0	0	Misma fecha (caso límite)
20000101	20250101	25	0	0	Años exactos

Orientaciones para la resolución

Análisis: Dos entradas `int` (fechas AAAAMMDD), determinación de mayor/menor, extracción de componentes, cálculo de diferencia con correcciones condicionales, salida formateada. Se recomienda elegir **uno** de los dos enfoques para implementar y verificar con todos los ejemplos de la tabla.

Diseño (Enfoque A): Variables: `dia_mayor`, `mes_mayor`, `anio_mayor`, `dia_menor`, `mes_menor`, `anio_menor` (`int`, entradas descompuestas); `dif_dia`, `dif_mes`, `dif_anio` (`int`, resultado). La lógica es clara: restar, corregir días si negativo, corregir meses si negativo. Requiere una forma de obtener los días de un mes dado (cadena de `if/elif` por ahora, función auxiliar en Semana 4).

Diseño (Enfoque B): Más compacto pero más sutil. La resta de enteros AAAAMMDD es una sola operación, pero la interpretación del resultado requiere entender que los «préstamos» entre dígitos operan en base 100, no en base 10. Este enfoque conecta directamente con el **complemento a dos** estudiado en la clase teórica (Semana 2): la misma idea de «complemento» para representar restas mediante sumas aparece aquí en base 100.

Codificación: En ambos enfoques, el punto crítico es determinar los días del mes anterior para la corrección. Esto requiere una cadena de `if/elif` que distinga los meses de 31, 30 y 28/29 días, incluyendo la regla de año bisiesto. Atención: cuando el mes mayor es enero (1), el «mes anterior» es diciembre (12) del año anterior.

Evaluación: Verificar con TODOS los ejemplos de la tabla, prestando especial atención a los casos con préstamo (día menor > día mayor) y al cruce de febrero bisiesto/no bisiesto. Un buen método de verificación: sumar la diferencia a la fecha menor y comprobar que se obtiene la fecha mayor.

Reflexión: complemento a 100 y complemento a dos

En la clase teórica de Semana 2 se estudia el complemento a dos para representar enteros negativos en binario: en lugar de restar, se suma el complemento. En este problema ocurre algo análogo en base 100: al restar AAAAMMDD como enteros, un préstamo entre la posición de días y la de meses opera en base 100 (no en base 30 ni 31). El valor 85 obtenido al restar día 10 – día 25 es el complemento a 100 de 15, que luego debe ajustarse a los días reales del mes. Esta analogía entre bases ayuda a internalizar el concepto de complemento.

Conexión con el Problema 3 de la Semana 1

La representación AAAAMMDD como entero, que en la Semana 1 se usó como parte entera de un número fraccionario, aquí se convierte en dato de entrada. El Enfoque B explota directamente las propiedades aritméticas de esta codificación posicional: la resta de enteros produce componentes separables por // y %. Esta reutilización ilustra un principio de diseño: las decisiones de codificación tienen consecuencias en los algoritmos que se pueden construir sobre ellas.

Problema 3: Conversión de decimal a número romano

Enunciado

Escribir un programa en Python que solicite al usuario un **número entero positivo entre 1 y 3999** y lo convierta a su representación en **números romanos**. El programa debe validar que el número esté en el rango válido y mostrar el resultado.

Fundamento matemático

El sistema de numeración romano usa siete símbolos básicos (I, V, X, L, C, D, M) y combinaciones sustractivas (IV, IX, XL, XC, CD, CM). El algoritmo propuesto se basa en una observación clave: **cada dígito del número decimal se convierte independientemente** según la posición que ocupa (unidades, decenas, centenas, millares). Cada posición tiene su propia tabla de equivalencias.

Paso 1: Descomposición en dígitos por posición

Dado un número entre 1 y 3999, se extraen sus cuatro dígitos posicionales mediante división entera y resto:

```
millares = numero // 1000
centenas = (numero % 1000) // 100
decenas = (numero % 100) // 10
unidades = numero % 10
```

Por ejemplo, para 2025: millares = 2, centenas = 0, decenas = 2, unidades = 5. Para 3999: millares = 3, centenas = 9, decenas = 9, unidades = 9.

Paso 2: Tablas de equivalencia por posición

Cada dígito (0–9) tiene una equivalencia romana distinta según la posición que ocupa, porque cada posición usa símbolos diferentes. Las cuatro tablas son:

Millares (dígito 0–3, con M):

Dígito	0	1	2	3
Romano	' '	M	MM	MMM

Centenas (dígito 0–9, con C, D, M):

Dígito	0	1	2	3	4	5	6	7	8	9
Romano	' '	C	CC	CCC	CD	D	DC	DCC	DCCC	CM

Decenas (dígito 0–9, con X, L, C):

Dígito	0	1	2	3	4	5	6	7	8	9
Romano	''	X	XX	XXX	XL	L	LX	LXX	LXXX	XC

Unidades (dígito 0–9, con I, V, X):

Dígito	0	1	2	3	4	5	6	7	8	9
Romano	''	I	II	III	IV	V	VI	VII	VIII	IX

Nótese el **patrón regular** en cada tabla: los dígitos 1–3 repiten el símbolo base, el 4 usa la forma sustractiva, el 5 usa el símbolo medio, 6–8 combinan medio + base, y el 9 usa sustractiva respecto al símbolo del siguiente orden. Este patrón es idéntico en las tres posiciones inferiores, solo cambian los símbolos.

Paso 3: Concatenación del resultado

Se presentan dos opciones de recorrido equivalentes:

Opción A — Significación descendente (millares → unidades): Se recorren los dígitos de izquierda a derecha y se **concatena a la derecha** del resultado:

```
romano = ''
romano = romano + equivalencia_millares    # romano = 'MM'
romano = romano + equivalencia_centenas    # romano = 'MM' + '' = 'MM'
romano = romano + equivalencia_decenas     # romano = 'MM' + 'XX' = 'MMXX'
romano = romano + equivalencia_unidades    # romano = 'MMXX' + 'V' = 'MMXXV'
```

Opción B — Significación ascendente (unidades → millares): Se recorren los dígitos de derecha a izquierda y se **concatena a la izquierda** del resultado:

```
romano = ''
romano = equivalencia_unidades + romano    # romano = 'V'
romano = equivalencia_decenas + romano     # romano = 'XX' + 'V' = 'XXV'
romano = equivalencia_centenas + romano    # romano = '' + 'XXV' = 'XXV'
romano = equivalencia_millares + romano    # romano = 'MM' + 'XXV' = 'MMXXV'
```

Ambas opciones producen el mismo resultado. La opción B es más natural cuando se extraen los dígitos de menor a mayor (primero unidades con % 10, luego decenas, etc.), ya que cada dígito se procesa en el orden en que se obtiene.

Ejemplos (casos de análisis y prueba)

Decimal	Millares	Centenas	Decenas	Unidades	Romano
1	0	0	0	1	I

Decimal	Millares	Centenas	Decenas	Unidades	Romano
4	0	0	0	4	IV
9	0	0	0	9	IX
42	0	0	4	2	XLII
399	0	3	9	9	CCCXCIX
444	0	4	4	4	CDXLIV
888	0	8	8	8	DCCCLXXXVIII
1582	1	5	8	2	MDLXXXII
2025	2	0	2	5	MMXXV
3999	3	9	9	9	MMMCMXCIX

Verificación manual para 1582 (opción A: descendente, concatenar a derecha)

```

numero = 1582

# Descomposición
millares = 1582 // 1000      = 1
centenas = (1582 % 1000) // 100 = 5
decenas  = (1582 % 100) // 10  = 8
unidades = 1582 % 10         = 2

# Conversión por posición (consulta en tablas)
romano = ''
# millares=1 → 'M'      → romano = '' + 'M'      = 'M'
# centenas=5 → 'D'      → romano = 'M' + 'D'      = 'MD'
# decenas=8  → 'LXXX'   → romano = 'MD' + 'LXXX' = 'MDLXXX'
# unidades=2 → 'II'     → romano = 'MDLXXX' + 'II' = 'MDLXXXII'

Resultado: MDLXXXII  ✓  (año de inicio del Calendario Gregoriano)

```

Verificación manual para 3999 (opción B: ascendente, concatenar a izquierda)

```

numero = 3999

# Descomposición (se extraen de menor a mayor)
unidades = 3999 % 10      = 9
decenas  = (3999 // 10) % 10 = 9
centenas = (3999 // 100) % 10 = 9
millares = 3999 // 1000    = 3

# Conversión por posición (concatenar a izquierda)
romano = ''

```

```
# unidades=9 → 'IX'      → romano = 'IX' + ''      = 'IX'
# decenas=9  → 'XC'      → romano = 'XC' + 'IX'    = 'XCIX'
# centenas=9 → 'CM'      → romano = 'CM' + 'XCIX'  = 'CMXCIX'
# millares=3 → 'MMM'     → romano = 'MMM' + 'CMXCIX' = 'MMMCMXCIX'
```

Resultado: MMMCMXCIX ✓ (máximo representable)

Orientaciones para la resolución

Análisis: Una entrada `int` (1–3999), una validación (`if numero < 1 or numero > 3999`), descomposición en 4 dígitos, consulta de 4 tablas de equivalencia (una por posición), concatenación de 4 cadenas, una salida `str`. Los casos sustractivos (4, 9) están incorporados en las tablas y no requieren lógica adicional.

Diseño: Variables: `numero` (`int`, entrada); `millares`, `centenas`, `decenas`, `unidades` (`int`, dígitos extraídos); `romano` (`str`, resultado construido por concatenación). La conversión de cada dígito requiere una cadena de `if/elif` con ramas para convertir dígitos que se representan como sumas con repeticiones de símbolos (1, 2, 3, 6, 7 y 8 en unidades, decenas, centenas y millares) y para los dígitos que se representan con un único símbolo o con restas (4, 5 y 9 en unidades, decenas y centenas); las representaciones de centenas, decenas y unidades tienen la misma estructura, solo cambian los símbolos.

Codificación: En la opción A (descendente), se concatena a la derecha: `romano = romano + equivalencia`. En la opción B (ascendente), a la izquierda: `romano = equivalencia + romano`. Ambas son correctas. La opción B combina naturalmente con la extracción ascendente (`% 10 + // 10` repetido), evitando calcular `% 1000` y `% 100` por separado.

Evaluación: Verificar con los 10 ejemplos de la tabla. Prestar especial atención a: 444 (sustractivo en las tres posiciones inferiores), 888 (la representación más larga posible para un dígito en cada posición), y 2025 (centenas = 0, que debe producir cadena vacía, no un símbolo).

Reflexión: ¿por qué 3999 como límite?

El sistema romano clásico no tiene símbolo para 5000. El valor 4000 requeriría un símbolo sustractivo respecto a 5000, que no existe, y la tabla de millares solo llega hasta 3 (MMM). Históricamente se usaron barras superiores (vinculum) para valores mayores, pero exceden el sistema estándar de siete símbolos.

Conexión con Arquitectura: sistemas posicionales

Este problema ilustra la diferencia entre un sistema posicional (decimal, donde el valor de un dígito depende de su posición) y un sistema aditivo-sustractivo (romano, donde los símbolos se suman o restan independientemente de su posición absoluta). La descomposición en dígitos conecta con la representación binaria vista en la clase teórica: el mismo algoritmo (dividir por la base, tomar el resto) sirve para extraer dígitos en cualquier base numérica.

Problema 4: Cardinalidad de un entero en palabras

Enunciado

Escribir un programa en Python que solicite al usuario un **número entero positivo** (sin límite superior práctico razonable, al menos hasta 999.999.999) y muestre su **cardinalidad en palabras** en español. Por ejemplo, 42 debe producir 'cuarenta y dos' y 1000000 debe producir 'un millón'.

Fundamento matemático

El sistema de numeración decimal posicional se estructura en **grupos de tres dígitos** (unidades, miles, millones) separados por puntos en la notación argentina. Cada grupo se convierte por separado y luego se concatena con la palabra del orden correspondiente.

Descomposición por grupos de tres dígitos

```
millones = numero // 1000000
miles = (numero % 1000000) // 1000
unidades = numero % 1000
```

Conversión de un grupo de tres dígitos (0–999)

Cada grupo se descompone en centenas, decenas y unidades:

```
centenas = grupo // 100   decenas = (grupo % 100) // 10   unidades = grupo % 10
```

Reglas del español para cardinales

Las principales reglas son:

Unidades (1–9): uno, dos, tres, cuatro, cinco, seis, siete, ocho, nueve. Caso especial: `uno` se convierte en `un` cuando precede a `mil` o `millón`.

Especiales (11–15): once, doce, trece, catorce, quince (nombres propios, no compuestos).

Decenas (10–29): diez, once...quince, dieciséis, diecisiete...diecinueve, veinte, veintiuno...veintinueve (una sola palabra).

Decenas (30–99): treinta, cuarenta...noventa + ' y ' + unidad. Ejemplo: treinta y dos.

Centenas: cien (exacto), ciento + resto, doscientos...novecientos. Casos irregulares: quinientos (no cincocientos) y setecientos, novecientos.

Miles: mil (no un mil) para 1000, pero dos mil, tres mil, etc. para valores mayores.

Millones: un millón (singular), dos millones (plural). Seguido de `de` cuando no hay miles ni unidades: un millón de (no un millón) en expresiones como «un millón de personas», pero para nuestro propósito (el número aislado) se omite el `de`.

Ejemplos (casos de análisis y prueba)

Número	Cardinalidad en palabras	Tipo de caso
1	uno	Mínimo
15	quince	Especial (11-15)
21	veintiuno	Veinti- compuesto
42	cuarenta y dos	Decena + unidad
100	cien	Centena exacta
101	ciento uno	Centena + unidad
500	quinientos	Centena irregular
999	novecientos noventa y nueve	Máximo de tres dígitos
1000	mil	Mil exacto (no 'un mil')
1001	mil uno	Mil + unidad
2025	dos mil veinticinco	Año actual
10000	diez mil	Decena de millar
100000	cien mil	Centena de millar
1000000	un millón	Millón singular
2000000	dos millones	Millón plural
1000001	un millón uno	Millón + unidad
999999999	novecientos noventa y nueve millones novecientos noventa y nueve mil novecientos noventa y nueve	Máximo

Verificación manual para 2025

```

numero = 2025
millones = 2025 // 1000000          = 0 (sin millones)
miles    = (2025 % 1000000) // 1000 = 2
unidades = 2025 % 1000              = 25

# Grupo miles = 2: 'dos' + ' mil' → 'dos mil'
# Grupo unidades = 25: caso 21-29 → 'veinticinco'
# Resultado: 'dos mil veinticinco' ✓

```

Orientaciones para la resolución

Análisis: Una entrada `int`, descomposición en grupos de tres dígitos, conversión de cada grupo a palabras (muchas ramas `if/elif`), ensamblado final con los conectores (`mil`, `millón/millones`). Este es el problema más complejo de la práctica: requiere un análisis exhaustivo de casos.

Diseño: Descomponer en subproblemas: (1) convertir un número de 0 a 999 a palabras, (2) aplicar eso a cada grupo, (3) ensamblar. Aunque sin funciones (Semana 4), el código se beneficia de comentarios que marquen claramente cada subproblema. Variables clave: `millones`, `miles`, `unidades` (int); `texto_millones`, `texto_miles`, `texto_unidades`, `resultado` (str).

Codificación: Los casos especiales (11–15, 16–19, 21–29, quinientos, setecientos, novecientos, cien vs ciento, mil vs un mil) requieren ramas condicionales explícitas. Prestar especial atención a los espacios entre palabras: el resultado no debe tener espacios al inicio, al final ni dobles.

Evaluación: Verificar con TODOS los ejemplos de la tabla, prestando especial atención a: los límites de cada rango (0, 1, 10, 11, 15, 16, 20, 21, 29, 30, 99, 100, 101, 999, 1000, 1001), las irregularidades (quinientos, setecientos, novecientos) y los conectores (' y ' solo entre decena y unidad).

Reflexión: complejidad vs. dificultad algorítmica

Este problema no requiere matemáticas avanzadas (solo divisiones y restos), pero es el más difícil de la práctica porque tiene muchos casos especiales. Esto ilustra que la complejidad de un programa no siempre proviene de las fórmulas sino de la cantidad de reglas del dominio. El análisis exhaustivo de casos (fase 1 de Pólya) es aquí más crítico que nunca.

Conexión con Arquitectura: representación vs. valor

Los Problemas 3 y 4 son inversamente simétricos: el Problema 3 convierte de una representación numérica (decimal) a otra (romano), mientras que el Problema 4 convierte de representación numérica a representación lingüística. En ambos casos, el valor matemático subyacente es el mismo; lo que cambia es el sistema de codificación. Este concepto es central en arquitectura de computadoras: un mismo dato tiene múltiples representaciones posibles (binario, decimal, ASCII, IEEE 754).

Notas finales

Estos cuatro problemas cubren los contenidos prácticos de la **Semana 2** del curso, adelantando estructuras condicionales (`if/elif/else`) de la Semana 3. Trabajan intensivamente con **aritmética modular** (`//` y `%`), **construcción de cadenas** (concatenación, multiplicación `str×int`), y **cadenas de decisiones** (`if/elif/else`). Los problemas están ordenados en complejidad creciente y presentan conexiones deliberadas: los Problemas 1–2 con fechas AAAAMMDD (enlazando con Semana 1) y los Problemas 3–4 con sistemas de representación.

Criterios de evaluación formativa

Además de los criterios de la Semana 1 (metodología de Pólya, modelo de programa, *literate programming*), esta semana se evaluarán especialmente: el **análisis exhaustivo de casos** (identificar todos los rangos y excepciones antes de codificar), el **uso correcto de condicionales** (sin ramas redundantes ni condiciones contradictorias), y la **traza manual** de los caminos de ejecución (verificar qué rama se toma para cada caso de prueba).

Para profundizar

(1) Implementar la conversión inversa del Problema 3: de romano a decimal; (2) extender el Problema 4 para manejar números ordinales (primero, segundo, tercero...); (3) combinar los Problemas 1 y 2: dado un par de fechas, mostrar cuántos días, semanas y días sueltos las separan, e indicar en qué día de la semana cae cada una; (4) ¿cómo se modificaría la fórmula de Zeller para fechas del calendario Juliano (anterior a 1582)?